RESEARCH WHITEPAPER

Measuring the Prompt Compression–Fidelity Tradeoff

Initial evidence from production-like agent traces and a research design for detecting degradation

FocusedObjective / PromptCompression

Preprint v0.1 | July 12, 2026

Open source: github.com/FocusedObjective/PromptCompression

Hosted service: compress.usagetap.com

Abstract

Prompt compression creates an optimization problem: remove enough input to reduce cost and latency, but not so much that downstream quality degrades. This paper reports an initial evaluation of FocusedObjective/PromptCompression, an open-source service built around LLMLingua-2 plus deterministic preprocessing, protected spans, structured-data handling, and role-aware message compression. We analyze four benchmark exports from two production-like workloads, FocusFit and DeliveryTower. The exports contain the original prompt, compressed prompt, token attribution, latency, and execution metadata.

Across 59 matched prompts, savings are real but workload-dependent. FocusFit reduced input tokens by approximately 5.3%, while DeliveryTower reduced them by approximately 13.7%. This spread shows why compression cannot be characterized by one global ratio: prompt composition, protected-data density, minimum segment gates, and learned-stage eligibility all affect the attainable reduction. Diagnostic comparisons also surface potential degradation signals—missing or reformatted values, altered signs or units, repeated-value count changes, and structure transformations—but these signals are not equivalent to semantic errors. One recent improvement, JSON protection, provides a useful intervention case study: it improved observed distinct numeric-literal and UUID retention in FocusFit while changing weighted savings by only 0.09 percentage points.

The current exports are sufficient to measure efficiency and identify candidate risk, but insufficient to establish semantic equivalence. Literal retention can miss paraphrase, relationships, negation, ordering, and instruction priority. Conversely, an exact-string metric can label a harmless transformation—such as `75%` becoming `75 %`—as a loss. The next benchmark must therefore measure downstream behavior. We propose a paired, blinded experiment that derives five evidence-grounded questions per prompt, runs the same downstream model against original and compressed contexts, and evaluates answer correctness, answerability, critical-fact retention, hallucination, structural validity, savings, and latency. The result is a general framework for locating the quality–efficiency frontier, not a test of JSON handling alone.

1. The question that matters

The product goal is not “remove as many tokens as possible.” It is:

> ***Reduce the prompt presented to a downstream model while preserving the facts, constraints, structure, and relationships required to produce the same useful answer.***

That definition creates two simultaneous objectives:

1. **Efficiency:** fewer input tokens, lower input cost, and—where prefill dominates—lower end-to-end latency.
1. **Fidelity:** no meaningful degradation in the downstream task, with especially strong guarantees for identifiers, numbers, constraints, code, tool messages, schemas, and structured data.

The second objective is harder. A compressed prompt may remain readable while silently losing a quota threshold, a negative sign, a candidate identifier, an array member, a stop condition, or the relationship between two facts. A benchmark that reports only token savings cannot detect those failures.

1.1 Why a few percentage points matter at client scale

One client generated 2.3 billion input tokens over 85 active days in the April–July observation window shown below. That is an average of approximately 27.1 million input tokens per active day. At this scale, modest reductions become material even when aggressive compression would be unacceptable.

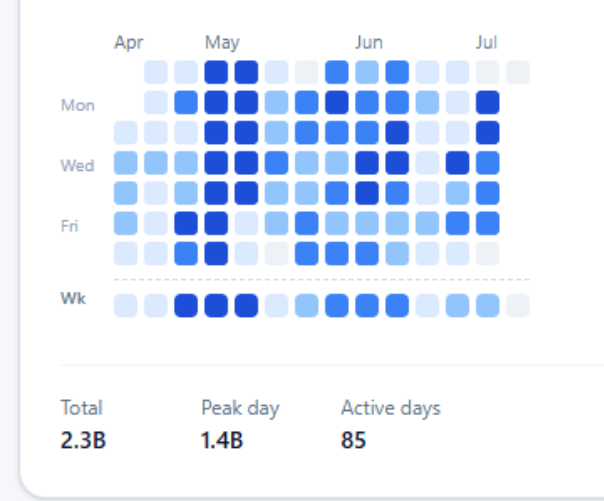

Figure 1. One client's input-token volume: 2.3B total across 85 active days; 1.4B on the peak day.

Scenario applied to 2.3B input tokens	Implied tokens avoided	Share of original input
1% reduction	23.0M	1.00%
FocusFit-like observed reduction	121.4M	5.28%
10% planning case	230.0M	10.00%
DeliveryTower-like observed reduction	314.4M	13.67%
15% upside case	345.0M	15.00%

If `P` is the effective price per million input tokens after model mix, prompt-caching discounts, and contract terms, the gross input-cost opportunity is `tokens avoided / 1,000,000 × P`. The two observed workload rates therefore correspond to approximately `121.4 × P` through `314.4 × P` over the

same 2.3B-token volume. This is an extrapolation, not a measured client saving: the client's prompt mix, cache eligibility, protection density, and achievable non-degrading compression rate must be measured call by call.

The correct business objective is not to maximize the percentage removed. It is to find the highest verified non-degrading rate for each workload. At 2.3B tokens, moving the safe frontier by even one percentage point represents another 23 million input tokens avoided.

2. System under evaluation

PromptCompression is an open-source HTTP service that uses LLMLingua-2 as its learned token-classification stage and surrounds it with deterministic controls. The current repository describes:

- deterministic normalization before model compression;
- protected spans for URLs, email addresses, money values, identifiers, constants, numbers, inline code, and selected constraints;
- role-aware chat compression that preserves non-user messages by default;
- JSON handling that can convert eligible data to TOON or protect it from probabilistic compression;
- tenant-authorized selective compression of allowlisted long string values inside tagged JSON;
- token attribution across deterministic and learned stages; and
- an evaluation suite for required substrings, forbidden substrings, protected structures, savings, and latency.

This layered design is important. LLMLingua-2 formulates prompt compression as token classification and reports strong task-level results across several benchmarks, but production agent prompts contain tool protocols, IDs, schemas, and tenant-specific constraints that are not adequately represented by natural-language compression scores alone [1]. The repository's protection layer is therefore part of the method being evaluated, not merely an implementation detail.

2.1 More than a call to LLMLingua-2

The system should be evaluated as a **compression and protection pipeline**, not as a wrapper that simply sends the whole prompt to LLMLingua-2. Raw token classification is well suited to compressible prose, but it has no inherent guarantee that a removed token is unimportant to a particular tenant, tool protocol, schema, or downstream question. PromptCompression first classifies and transforms content, applies deterministic savings where it can, removes unsafe content from the learned model's reach, and then uses LLMLingua-2 on eligible prose segments.

Layer	Primary function	Intended advantage over raw LLMLingua-2
Whitespace normalization	Trims trailing whitespace and collapses excessive blank lines; optional strict mode collapses repeated interior spaces only in prose that does not look like indentation, a table, list, blockquote, YAML, or ASCII alignment	Low-risk deterministic savings without asking a model to judge formatting tokens
HTML compaction	Converts eligible full HTML documents to extracted Markdown when the input is large enough and the conversion clears a minimum savings gate; preserves exactness-sensitive HTML and code-bearing blocks	Removes tags, navigation, scripts, styles, and presentation overhead while retaining readable content
JSON and TOON handling	Detects valid JSON; protects small or exactness-sensitive JSON; converts eligible medium/large JSON to TOON only when beneficial; optionally minifies safe fallbacks	Compresses structured data without probabilistic deletion of keys, types, values, or relationships
Protected and verbatim spans	Shields code fences, explicit nocompress regions, exact fixtures, tool exchanges, URLs, identifiers, money, constraints, and other critical spans	Creates fail-closed safety boundaries for content whose exact form matters
Role-aware message handling	Preserves stable system/developer and other non-user messages by default while targeting user-supplied context	Protects instruction hierarchy and downstream prompt caching
LLMLingua-2 stage	Applies learned token classification only to eligible, sufficiently large prose segments	Captures additional semantic redundancy that deterministic transforms cannot remove

The layers solve different problems. Whitespace, HTML, and structured-data transforms can produce savings with rules that are directly testable. Protection rules trade some compression for lower catastrophic-error risk. LLMLingua-2 supplies flexible semantic compression for the remaining prose. The research question is therefore not only “Does LLMLingua-2 work?” but “Does the full layered pipeline move the compression–fidelity frontier beyond raw LLMLingua-2?”

Deterministic does not mean automatically correct. HTML-to-Markdown conversion can omit content that a later question needs; TOON changes representation; whitespace rules can damage alignment if

misclassified. The advantage is auditability: each transform has an explicit applicability gate, measurable savings, and invariants that can be tested without an LLM judge.

3. Data and methods

3.1 Benchmark exports

We analyzed four CSV exports supplied by the project author:

- FocusFit before JSON protection: 39 rows;
- FocusFit after JSON protection: 39 rows;
- DeliveryTower before JSON protection: 20 rows; and
- DeliveryTower after JSON protection: 25 rows.

The filenames include “25,” but the observed row counts are 39, 39, 20, and 25. The analysis uses the data actually present rather than inferring sample size from filenames. Matching on `callId` and `messageIndex` yields 39 paired FocusFit prompts and 20 paired DeliveryTower prompts, or 59 paired prompts in total. The five additional DeliveryTower “after” rows are reported separately and are not used for paired before/after claims.

The traces were generated between July 5 and July 10, 2026. FocusFit includes GPT-5.2 and gpt-5-mini target-model labels; DeliveryTower includes Claude Haiku 4.5, gpt-5-mini, and GPT-5.4-mini labels. Every row reports `model_force`; FocusFit contains both `deterministic_plus_model` and `unchanged` paths, while DeliveryTower uses `deterministic_plus_model` throughout.

3.2 Metrics

For each export we calculated:

- total original and compressed tokens;
- weighted token reduction, `(original – compressed) / original`;
- mean and median row-level savings;
- learned-stage execution and fallback counts;
- exact retention of distinct numeric literals and UUIDs within each prompt; and
- occurrence-sensitive retention using the repository's protected-span patterns.

All before/after comparisons are paired where possible. Exact retention is intentionally treated as a diagnostic rather than a semantic score. It is strict about formatting, insensitive to paraphrase, and does not prove that a retained token remains attached to the correct entity or field.

4. Initial findings on the compression–fidelity balance

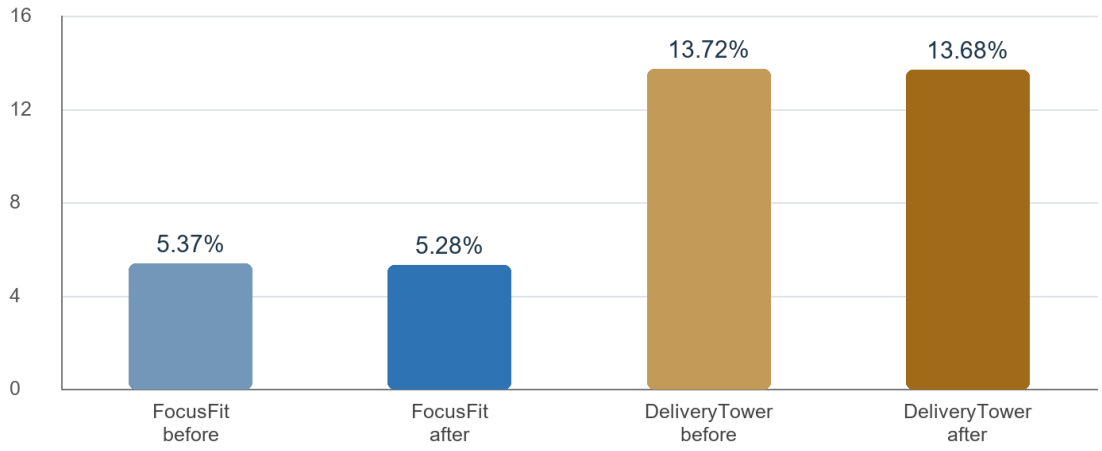

Figure 2. Weighted token reduction in the matched benchmark cohorts.

4.1 Compression savings are meaningful but workload-dependent

Cohort	Rows	Original tokens	Compressed tokens	Tokens saved	Weighted reduction
FocusFit, before JSON protection	39	318,770	301,654	17,116	5.37%
FocusFit, after JSON protection	39	318,770	301,945	16,825	5.28%
DeliveryTower, before (matched subset)	20	18,333	15,818	2,515	13.72%
DeliveryTower, after (matched subset)	20	18,333	15,826	2,507	13.67%
DeliveryTower, after (full export)	25	21,217	18,353	2,864	13.50%

The difference between workloads is more important than the small version-to-version change. DeliveryTower consistently saves roughly 13.5%–13.7%, while FocusFit saves roughly 5.3% on a far larger token base. FocusFit's median row-level savings is 0% because the learned stage ran on only 15 of 39 rows and many rows followed the unchanged path. A single global “compression rate” would conceal this mixture.

4.2 Efficiency metrics alone do not reveal degradation

Token reduction measures what was removed, not whether it was safe to remove. The current traces contain several classes of fidelity signal: exact values, identifiers, signs and units, repeated occurrences,

constraints, and structured records. Each class needs its own measurement because aggregate lexical similarity can remain high while a single critical fact changes.

Strict literal comparison illustrates the problem. It can detect that a value no longer appears in exactly the same form, but it cannot tell whether the value was safely reformatted, validly represented in a different structure, detached from its field, or genuinely deleted. Likewise, high retention of words or identifiers does not prove that relationships and instruction priority survived. These diagnostics are useful for triage; downstream behavior is the primary quality endpoint.

4.3 JSON protection as one intervention case study

FocusFit is the cleanest paired test of the JSON-protection change because all 39 original prompts are byte-identical across the two exports.

Diagnostic	Before	After	Change
Distinct numeric literals retained	4,094 / 4,103 (99.78%)	4,103 / 4,103 (100.00%)	+9 row-level distinct instances
UUIDs retained	616 / 621 (99.19%)	621 / 621 (100.00%)	+5 instances
Compressed tokens	301,654	301,945	+291 tokens
Weighted token reduction	5.369%	5.278%	−0.091 percentage points

This is a favorable trade on these traces: the protected version removes the observed distinct-number and UUID omissions while giving back fewer than one token per thousand original tokens. The result supports the design decision to shield structured data before learned compression.

It does **not** show that all structure is preserved. JSON may be safely transformed to TOON, which means byte identity and JSON syntax are not expected in every eligible block. The correct structural test is a typed round trip: parse the original, decode the transformed representation, and compare keys, types, values, array order, multiplicity, and relationships according to the applicable policy.

4.4 Strict literal matching exposes both risks and false alarms

Occurrence-sensitive matching with the repository's own protected-span patterns retained 7,522 of 8,331 numeric occurrences (90.3%) in FocusFit in both versions, even though the protected version retained every distinct numeric literal at least once per prompt. This discrepancy can arise when repeated values are removed, reformatted, or represented differently. It matters because repetition can be redundant, but it can also encode array multiplicity or separate records that happen to share a value.

DeliveryTower also produced exact-string misses that were unchanged by JSON protection. Manual inspection found examples such as `75%` becoming `75 %`: the strict matcher calls this a lost literal, although the visible meaning is preserved. Other misses include signed values such as `+64`, where removal of the sign may or may not be harmless depending on whether nearby text still identifies additions versus deletions. The lesson is two-sided:

- normalized literal metrics are needed to avoid counting formatting-only changes as semantic failures; and
- critical signed values, negations, units, and field associations need dedicated tests rather than generic token presence checks.

4.5 Latency is not yet attributable

Observed FocusFit mean compression time increased from 1.27 seconds before protection to 1.94 seconds after protection; the median moved from 171 ms to 189 ms. These are descriptive measurements, not causal estimates. The benchmark was not randomized, system load was not controlled, and phase-level diagnostics themselves add work. Future latency studies should warm the model, randomize condition order, repeat each prompt, separate cold-start from steady-state runs, and report phase-level medians and tail latency.

4.6 The current data cannot answer the semantic question

The exports contain prompts and compression diagnostics but no downstream answers, reference answers, question set, human judgments, or calibrated judge scores. They can show savings and surface suspicious transformations; they cannot establish that a user would receive the same answer. The most defensible conclusion is therefore:

> ***The system achieves measurable, workload-dependent compression and recent protection work improves some observable fidelity proxies. The effect of compression on downstream task quality remains unproven.***

5. Proposed experiment: five questions per prompt

The next evaluation should turn every prompt pair into a small, evidence-grounded exam. With the current 59 paired prompts, five questions per prompt would produce 295 paired question trials before model repetitions.

5.1 Question taxonomy

Each prompt receives one question from each category:

1. **Exact fact or identifier.** Requires an ID, name, date, URL, file path, status, or other atomic value. Example: “What is the sourcing run ID?”
2. **Number, unit, or constraint.** Requires a threshold, count, signed change, deadline, cost, percentage, or explicit prohibition. Example: “At what usage percentage should the alert fire, and how often?”
3. **Relationship or state transition.** Requires associating two or more facts. Example: “Which candidates moved to REVIEWED, and what grades did they receive?”
4. **Intent, procedure, or next action.** Tests whether instructions and priority survived. Example: “What should happen next, and what stopping condition must be checked first?”
5. **Negative-control or unanswerable question.** The correct response is that the prompt does not provide the requested fact. This detects hallucination and false answerability. Example: “What was the result of query q4?” when q4 has not run.

Questions should be generated from the **original** prompt, but a human reviewer must verify each reference answer and mark the minimal evidence span or structured-data path that supports it. To reduce wording bias, question generation should not simply copy the source sentence. A second reviewer should reject ambiguous questions, questions answerable from world knowledge alone, and questions whose answer depends on information outside the captured prompt.

5.2 Conditions

For each question, run the same downstream model under a layered ablation design:

- **Original:** uncompressed prompt plus question;
- **Raw LLMLingua-2:** the learned compressor applied without PromptCompression's deterministic transforms or protection suite;
- **Deterministic-only:** whitespace, HTML, JSON/TOON, and protection logic with the learned stage disabled;
- **Full pipeline:** deterministic suite plus LLMLingua-2 at the candidate aggressiveness;
- **Leave-one-layer-out ablations:** full pipeline minus whitespace normalization, HTML compaction, JSON/TOON handling, or protected spans, evaluated on prompts where that layer applies; and
- **Challengers:** alternate aggressiveness values and a quality-oriented larger LLMLingua-2 checkpoint.

The raw LLMLingua-2 baseline is essential: it quantifies whether the surrounding suite adds safety, additional deterministic savings, or both. Deterministic-only versus full-pipeline isolates the learned stage's incremental contribution. Leave-one-layer-out comparisons show which safeguards pay for themselves and which consume too much of the compression budget. Report these effects by content stratum—prose-heavy, HTML-heavy, JSON-heavy, tool-protocol, and mixed—because averaging an inapplicable layer across all prompts will dilute its contribution.

Keep the system prompt, tools, model version, decoding parameters, and output schema fixed. Randomize condition order and hide condition labels from graders. Use deterministic decoding when available; otherwise run at least three replicates per prompt-question-condition and model the replicate variance. Evaluate at least two downstream model families so the compressor is not optimized around one model's ability to reconstruct missing context.

5.3 Scoring

The primary endpoint should be paired answer correctness. Each answer receives:

- **Exact correctness:** exact match or normalized field-aware match for IDs, numbers, dates, enums, and short answers;
- **Semantic correctness:** 0–4 rubric scored by a blinded judge, with 4 meaning fully correct and complete;
- **Evidence support:** whether every material claim is supported by the supplied prompt;
- **Answerability:** correct answer, correct abstention, false abstention, or hallucinated answer;
- **Critical error flag:** wrong identifier, wrong sign/unit, inverted negation, omitted hard constraint, wrong state transition, or invalid tool/schema output; and
- **Format/structure validity:** parse success and schema/policy invariants for structured outputs.

Automated judges should not be the sole authority. Human reviewers should score all critical disagreements and a random 20% audit sample. Reviewer agreement should be reported. The judge should see the question, reference answer, evidence, and candidate answer—but not whether the candidate used original or compressed context.

5.4 Statistical analysis

This is a paired non-inferiority problem, not a contest for a higher average score. Before the run, choose a product-risk-based margin—for example, no more than a 2 percentage-point reduction in fully correct answers—and a stricter near-zero tolerance for critical structural failures.

Report:

- the paired difference in fully correct answer rate;
- a 95% confidence interval from a bootstrap clustered by prompt;
- McNemar's test or an exact paired test for binary pass/fail reversals;
- error rates by question category, workload, prompt length, savings band, and protected-data density;
- severe-error counts with every case reviewed manually; and
- the efficiency-quality frontier rather than a single aggregate score.

The current 295 question pairs are appropriate for a pilot, but clustering five questions within each prompt reduces the effective sample size. Use the pilot to estimate disagreement and intraprompt correlation, then power the confirmatory study. A reasonable next target is at least 200 diverse prompts (1,000 questions) across multiple tenants, prompt lengths, and data types, with a held-out regression set that is never used to tune protection rules.

6. Structural and adversarial test suite

Question answering should be complemented by deterministic invariants. For every structured segment:

1. detect whether the original is valid JSON, JSON Schema, a tool call/result, or an exact fixture;

2. apply the configured transformation;

3. parse or decode the result;

4. compare key sets, types, scalar values, array length and order, duplicate-key behavior, and required field paths;

5. verify occurrence counts for critical IDs and values; and

6. fail closed to verbatim protection when the transformation cannot be proven safe.

The adversarial set should deliberately include:

- negative and explicitly signed numbers (`-19`, `+64`), percentages, currency, units, and scientific notation;
- repeated values whose multiplicity matters;
- negation and exception clauses (`must not`, `unless`, `only if`);
- near-duplicate IDs and values differing by one character;
- nested arrays, empty values, nulls, booleans, escaped strings, and duplicate JSON keys;
- JSON embedded inside JSON strings and tool-call arguments;
- code, SQL, stack traces, Markdown tables, and exact-output templates;
- conflicting facts at different positions in long prompts; and
- prompt-injection text inside otherwise protected data.

Every production regression should record the compressor version, model checkpoint, tokenizer, tenant policy, aggressiveness, deterministic transforms, random seed where applicable, and downstream model version.

7. Recommended development sequence

Phase 1: Make the current benchmark auditable

- Persist a stable `prompt_pair_id` across every before/after run.
- Export configuration and version metadata with each row.
- Add normalized protected-literal comparisons by type.
- Add typed JSON/TOON round-trip checks and occurrence-count checks.
- Separate steady-state latency from cold-start and diagnostic overhead.

Phase 2: Add paired downstream evaluation

- Create and human-verify the five-question set for the 59 current pairs.
- Run original and compressed conditions blindly with fixed decoding.
- Review every answer reversal and every critical error.
- Publish prompt-level results or redacted reproducible fixtures where customer data prevents release.

Phase 3: Tune the quality–efficiency frontier by failure class

- Fix signed-number, negation, multiplicity, field-association, instruction-priority, and answerability failures.
- Compare deterministic-only, current BERT-base, and a quality-oriented larger checkpoint.
- Choose aggressiveness per workload and protected-data density.
- Maintain a locked holdout set to prevent evaluation overfitting.

Phase 4: Confirm non-inferiority

- Expand to at least 200 diverse prompts.
- Preregister primary metrics and margins.
- Report confidence intervals, critical failures, and efficiency-quality curves.
- Require a release gate: no structural invariant failures and demonstrated non-inferiority on the locked set.

8. Threats to validity

This initial study is limited in several ways:

- It contains only two workloads and 59 matched prompts.
- The workloads have different sizes, content, and learned-stage execution rates.
- Five DeliveryTower after-protection rows have no before counterpart.

- Token counts use the service's configured estimator and may differ from the downstream provider's billable tokenizer.
- Exact literal checks can both miss semantic corruption and overcount harmless formatting changes.
- The dataset contains no downstream outputs or human semantic labels.
- Latency conditions were not randomized or controlled.
- The prompts may contain repeated or correlated agent traces, reducing effective sample diversity.
- Protection rules and benchmark fixtures live in the same repository, creating a risk of overfitting unless a locked external holdout is added.

9. Conclusion

The initial data supports two practical conclusions. First, PromptCompression is achieving non-trivial token reduction on real agent traces, but the attainable savings vary sharply by workload. Second, fidelity must be evaluated as a portfolio of risks rather than a single similarity score. Exact facts, signed values, constraints, relationships, state transitions, answerability, and structured data fail in different ways. JSON protection is one encouraging example of improving a specific risk class with little effect on savings, not the central definition of quality.

The data does not yet justify the stronger claim that meaning is preserved. That claim requires a behavioral test. The proposed five-question protocol turns each prompt into a paired downstream evaluation, while deterministic invariants cover failure modes that language-model judging should never be asked to excuse. The goal is to publish an efficiency–quality curve: how much compression each workload can tolerate before meaningful degradation appears, which content classes fail first, and which safeguards move that frontier outward.

References

1. Pan, Z., et al. “LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression.” Findings of ACL, 2024. https://aclanthology.org/2024.findings-acl.57/

2. Bai, Y., et al. “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding.” ACL, 2024. https://aclanthology.org/2024.acl-long.172/

3. Es, S., et al. “RAGAS: Automated Evaluation of Retrieval Augmented Generation.” EACL System Demonstrations, 2024. https://aclanthology.org/2024.eacl-demo.16/

4. FocusedObjective. “PromptCompression.” Source repository and project documentation. https://github.com/FocusedObjective/PromptCompression

5. FocusedObjective. “PromptCompression hosted service.” https://compress.usagetap.com

Appendix A. Minimum evaluation record

Each prompt-question trial should store:

- `prompt_pair_id`, workload, tenant-safe category, and prompt length;
- original and compressed prompt hashes;
- compressor commit, configuration, model checkpoint, and tokenizer;
- question category, question, reference answer, and evidence spans/paths;
- downstream model, system prompt hash, decoding settings, and replicate number;
- raw answer, normalized answer, exact score, rubric score, and critical-error tags;
- structured-data invariant results;
- original/compressed tokens, provider-token estimates, cost, and latency phases; and
- blinded reviewer/judge identifiers and adjudication status.

Appendix B. Release gate proposal

A candidate compressor release should pass all of the following:

1. Zero JSON/TOON round-trip, tool-protocol, schema, or exact-fixture invariant failures on the locked set.

2. No unresolved critical error involving identity, sign, unit, negation, hard constraint, or state transition.

3. The lower bound of the clustered 95% confidence interval for the paired fully-correct rate is above the preregistered non-inferiority margin.

4. Savings and latency meet workload-specific targets; quality is never traded for a global compression target.

5. Every newly discovered production failure becomes a minimal regression fixture plus a question-answer case.